ASSIGNMENT-15.3

Name: Chirra Jyothika

Ht.no:2303A51280

Batch:05

Task 1: Real-Time Application: Online Food Ordering API
Scenario:

Design a simple API for an online food ordering system.

Requirements:

• Endpoints required:

• GET /menu → List available dishes

• POST /order → Place a new order

• GET /order/{id} → Track order status

• PUT /order/{id} → Update an existing order (e.g., change items)

• DELETE /order/{id} → Cancel an order

• AI should generate:

• REST API code

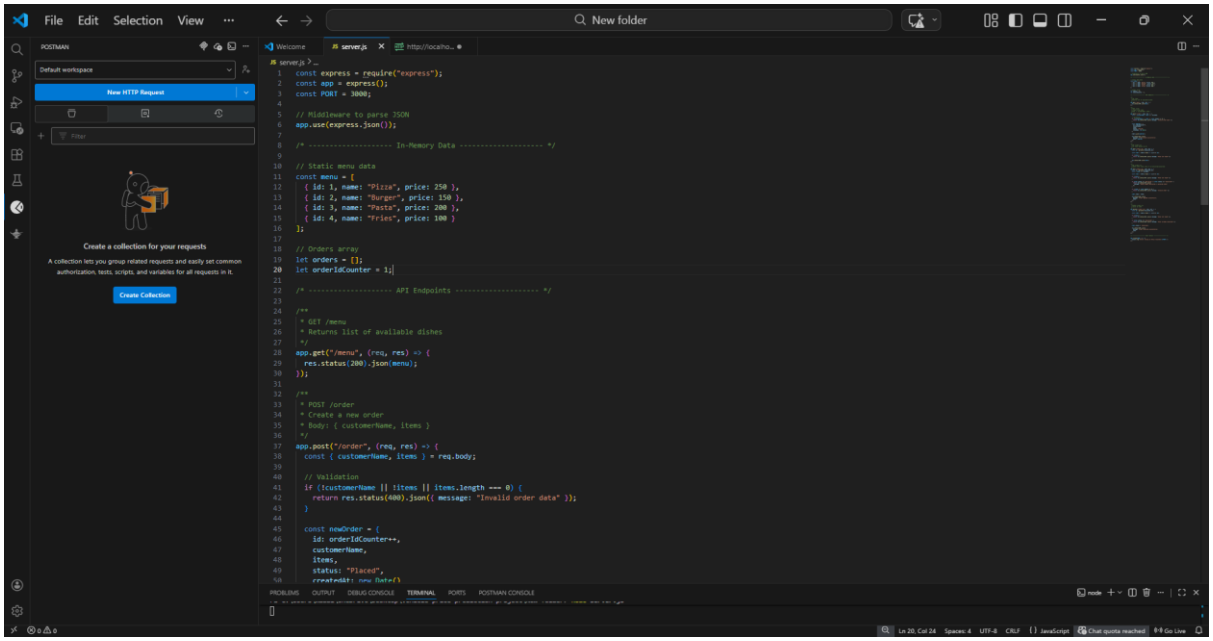
• Suggested improvements (like authentication, pagination)

Prompt:

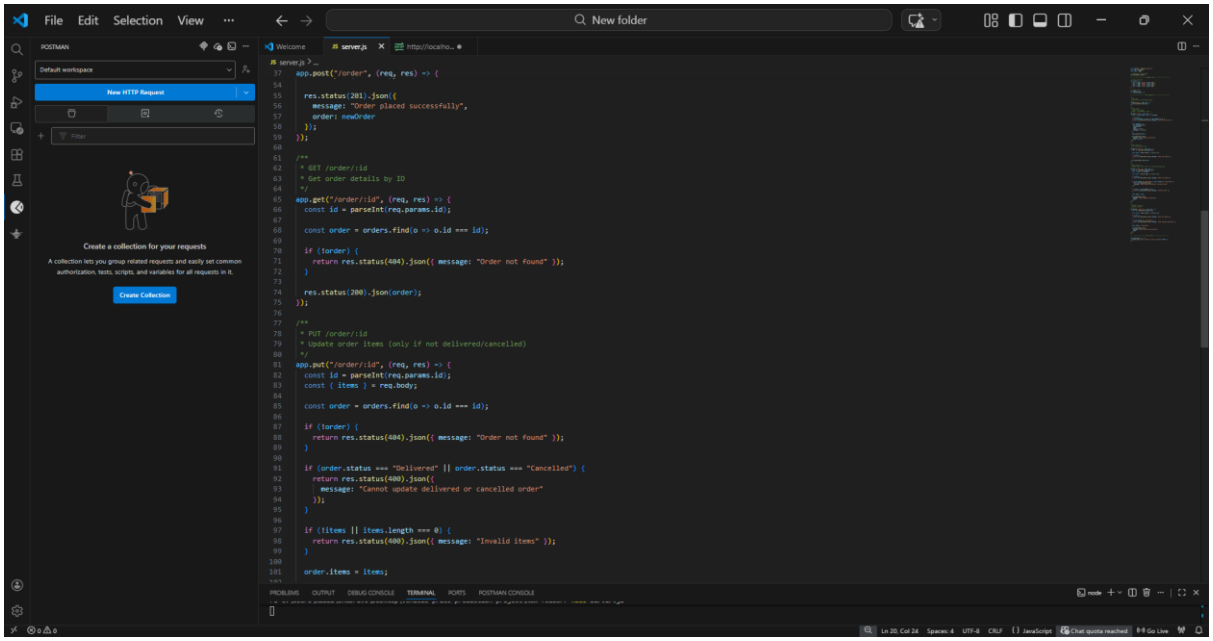
```
Create a Node.js Express REST API for an online food
ordering system.
Requirements: - GET /menu → return list of dishes - POST
/order → create new order with items - GET /order/:id →
get order status - PUT /order/:id → update order items -
DELETE /order/:id → cancel order
Use in-memory arrays. Add proper comments for each
endpoint.
```

Code:

server.js

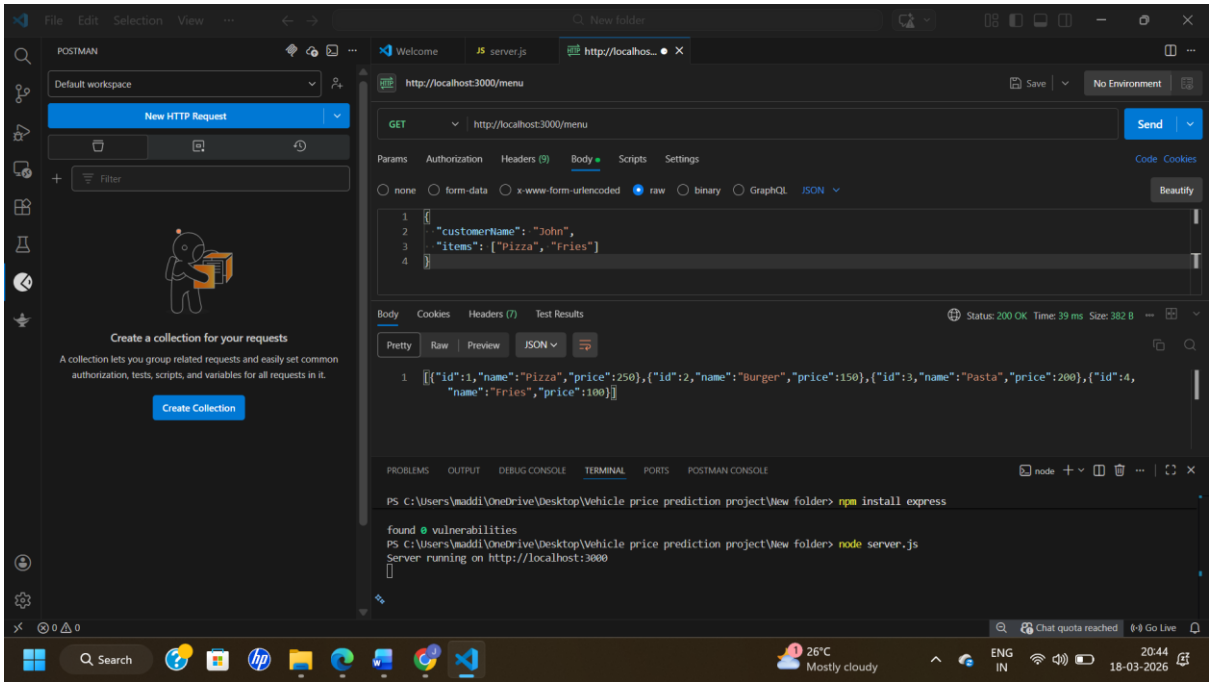


```
const express = require("express");
const app = express();
const PORT = 3000;

// Middleware to parse JSON
app.use(express.json());

/* ------------------- In-Memory Data ------------------- */

// Static menu data
const menu = [
  { id: 1, name: "Pizza", price: 250 },
  { id: 2, name: "Burger", price: 150 },
  { id: 3, name: "Pasta", price: 200 },
  { id: 4, name: "Fries", price: 100 }
];

// Orders array
let orders = [];
let orderIdCounter = 1;

/* ------------------- API Endpoints ------------------- */

/**
 * GET /menu
 * Returns list of available dishes
 */
app.get("/menu", (req, res) => {
  res.status(200).json(menu);
});

/**
 * POST /order
 * Create a new order
 * Body: { customerName, items }
 */
app.post("/order", (req, res) => {
  const { customerName, items } = req.body;

  // Validation
  if (!customerName || !items || items.length === 0) {
    return res.status(400).json({ message: "Invalid order data" });
  }

  const newOrder = {
    id: orderIdCounter++,
    customerName,
    items,
    status: "Placed",
    createdAt: new Date()
```

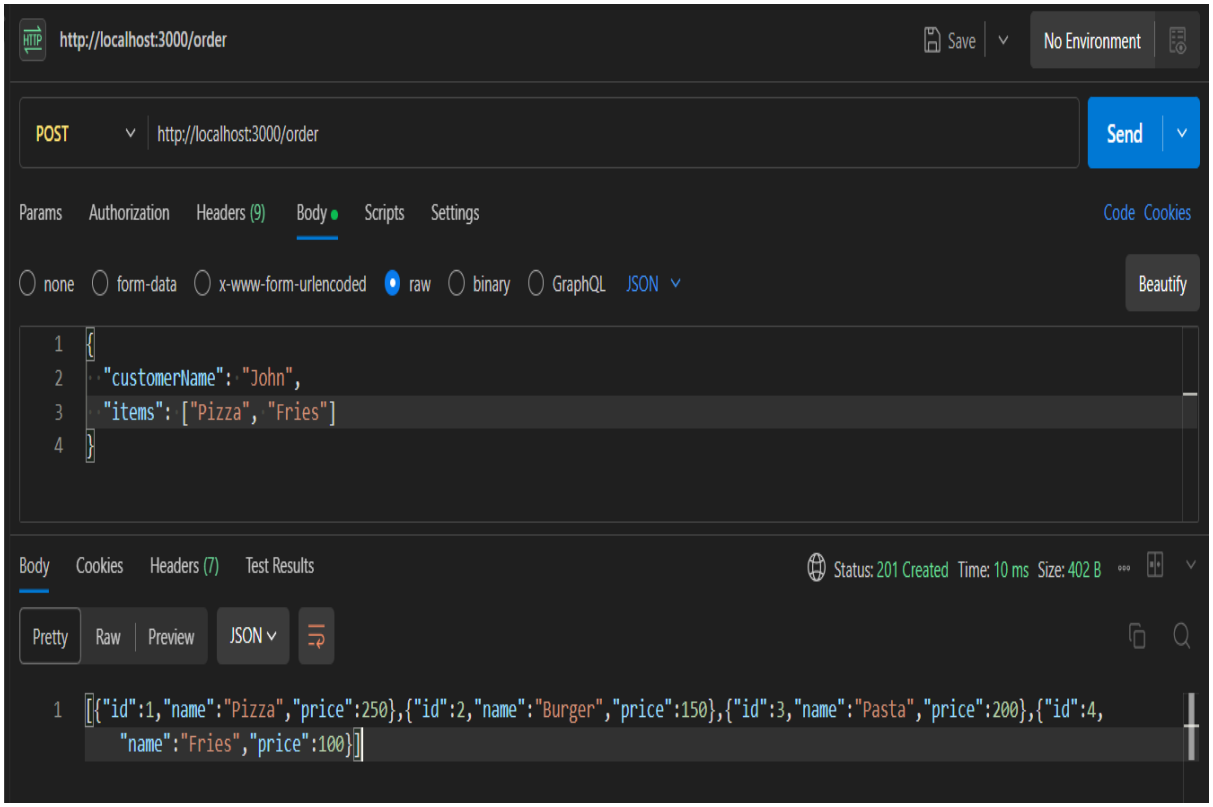


```
app.post("/order", (req, res) => {

  res.status(201).json({
    message: "Order placed successfully",
    order: newOrder
  });
});

/**
 * GET /order/:id
 * Get order details by ID
 */
app.get("/order/:id", (req, res) => {
  const id = parseInt(req.params.id);

  const order = orders.find(o => o.id === id);

  if (!order) {
    return res.status(404).json({ message: "Order not found" });
  }

  res.status(200).json(order);
});

/**
 * PUT /order/:id
 * Update order items (only if not delivered/cancelled)
 */
app.put("/order/:id", (req, res) => {
  const id = parseInt(req.params.id);
  const { items } = req.body;

  const order = orders.find(o => o.id === id);

  if (!order) {
    return res.status(404).json({ message: "Order not found" });
  }

  if (order.status === "Delivered" || order.status === "Cancelled") {
    return res.status(400).json({
      message: "Cannot update delivered or cancelled order"
    });
  }

  if (!items || items.length === 0) {
    return res.status(400).json({ message: "Invalid items" });
  }

  order.items = items;
```

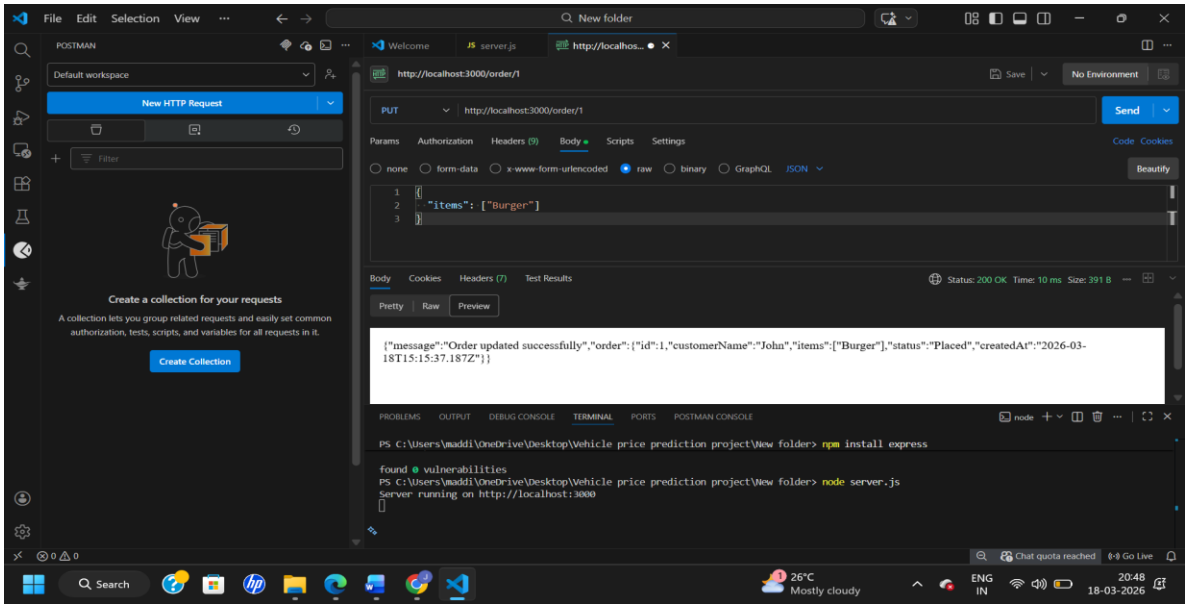
Get:


POSTMAN
Default workspace
New HTTP Request
Create a collection for your requests
A collection lets you group related requests and easily set common authorization, tests, scripts, and variables for all requests in it.
Create Collection
http://localhost:3000/menu
GET
http://localhost:3000/menu
Send
No Environment
Params
Authorization
Headers (9)
Body
Scripts
Settings
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
{
"customerName": "John",
"items": ["Pizza", "Fries"]
}
Status: 200 OK Time: 39 ms Size: 382 B
[{"id":1,"name":"Pizza","price":250},{"id":2,"name":"Burger","price":150},{"id":3,"name":"Pasta","price":200},{"id":4,"name":"Fries","price":100}]
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
POSTMAN CONSOLE
PS C:\Users\maddi\OneDrive\Desktop\Vehicle price prediction project\New folder> npm install express
found 0 vulnerabilities
PS C:\Users\maddi\OneDrive\Desktop\Vehicle price prediction project\New folder> node server.js
Server running on http://localhost:3000

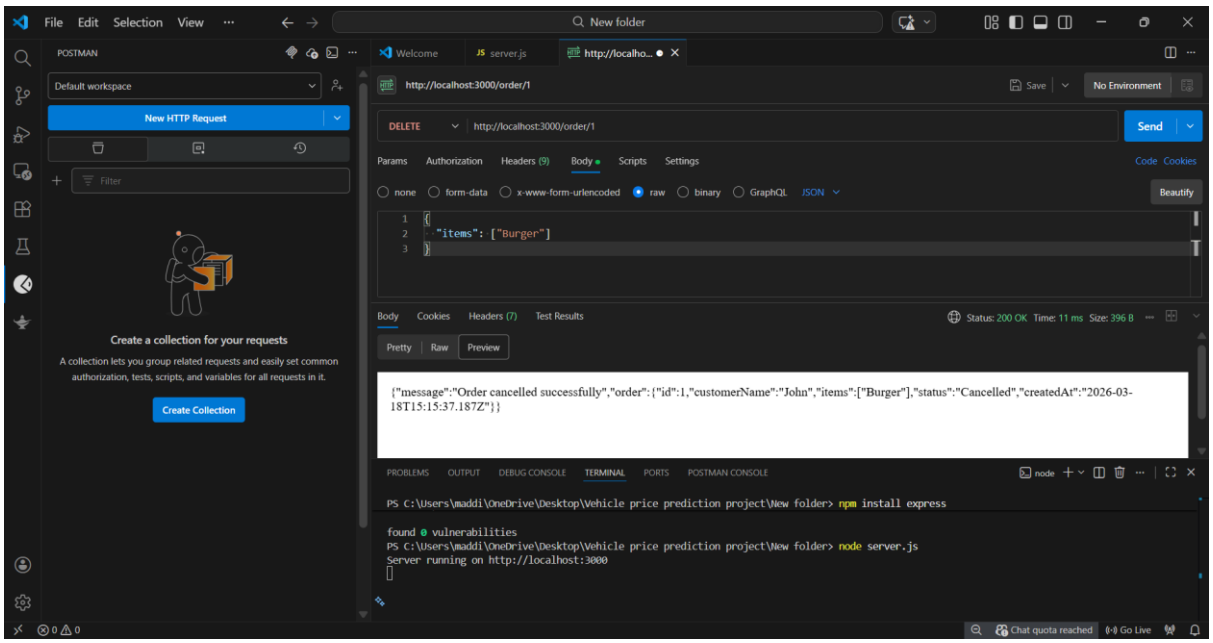
Post:


http://localhost:3000/order
Save
No Environment
POST
http://localhost:3000/order
Send
Params
Authorization
Headers (9)
Body
Scripts
Settings
Code
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
{
"customerName": "John",
"items": ["Pizza", "Fries"]
}
Body
Cookies
Headers (7)
Test Results
Status: 201 Created Time: 10 ms Size: 402 B
Pretty
Raw
Preview
JSON
[{"id":1,"name":"Pizza","price":250},{"id":2,"name":"Burger","price":150},{"id":3,"name":"Pasta","price":200},{"id":4,"name":"Fries","price":100}]

Put:



Delete:



Explanation:

This API simulates a basic online food ordering system using REST principles.

The GET /menu endpoint retrieves available dishes.

The POST /order endpoint allows users to place a new order by sending selected items.

The **GET /order/:**id endpoint helps in tracking the status of an order. The **PUT /order/:**id endpoint is used to update order details such as items.

The **DELETE /order/:id** endpoint cancels an existing order.

All data is stored in an in-memory array, making it simple and fast for testing.

Task 2: Employee Payroll API
Scenario:

Create an API for managing employees and their salaries.

Instructions:

• Endpoints required:

• GET /employees → List all employees

• POST /employees → Add a new employee with salary details

• PUT /employees/{id}/salary → Update salary of an employee

• DELETE /employees/{id} → Remove employee from system

• Use AI to:

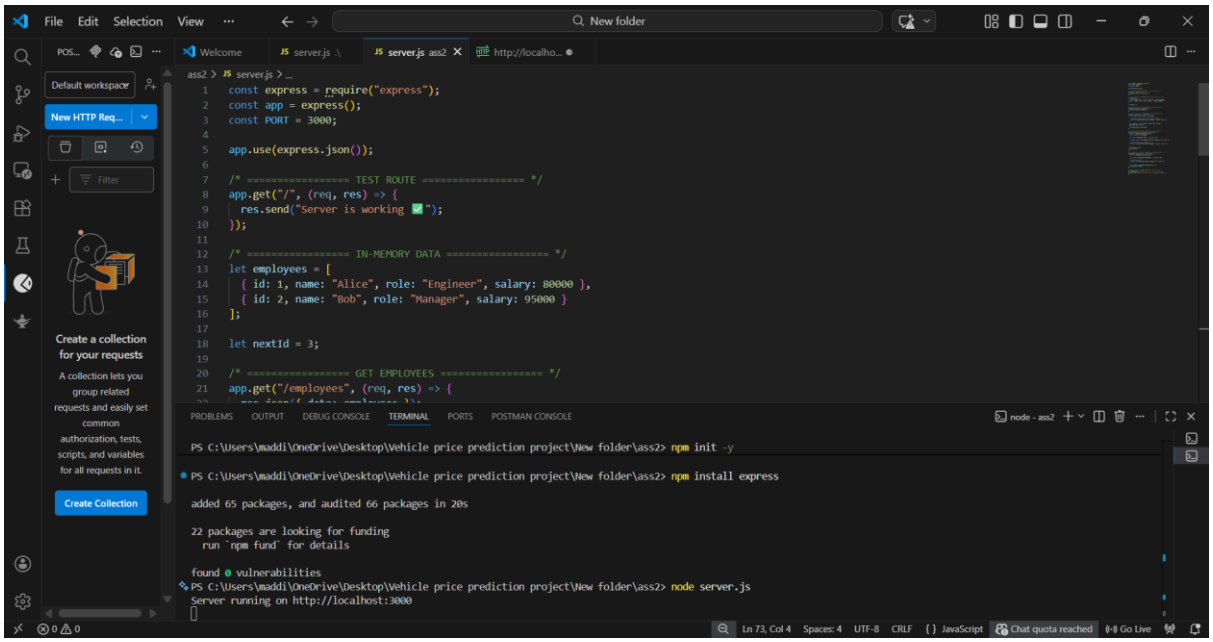
• Suggest data model structure.

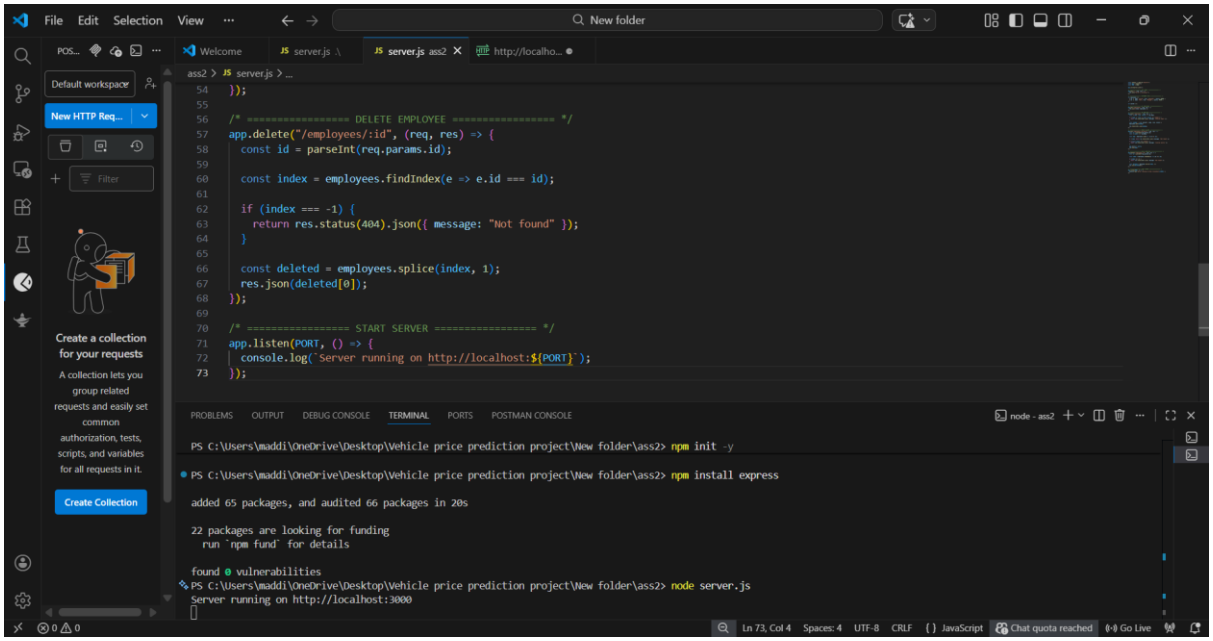
• Add comments/docstrings for all endpoints.

Prompt:

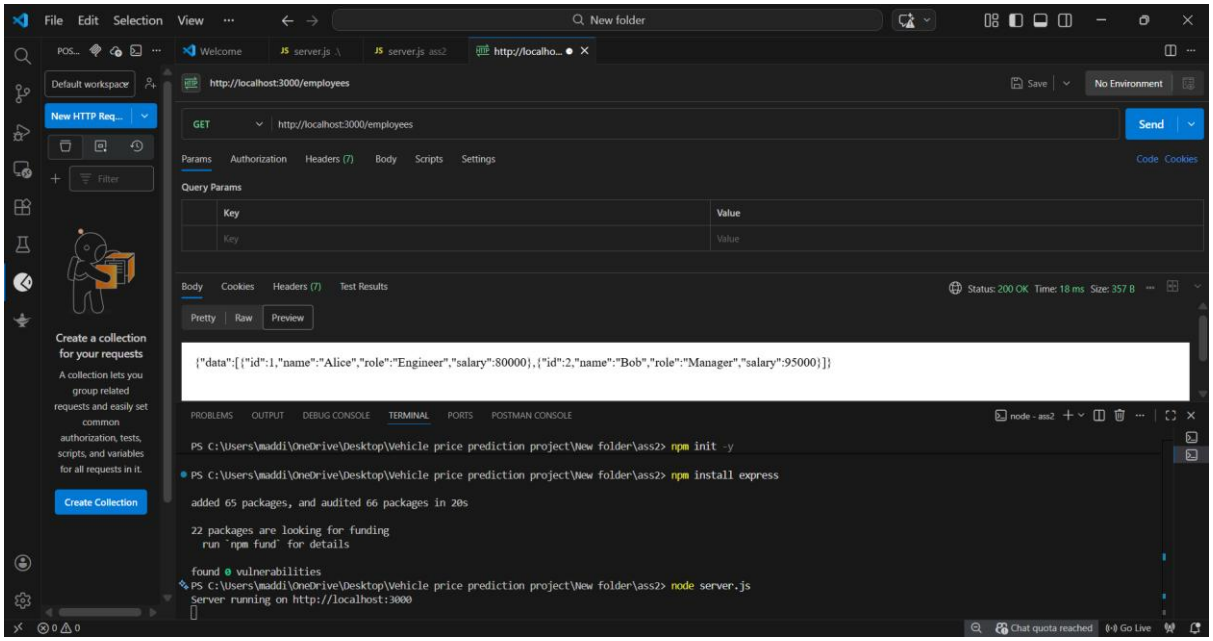
Create a Node.js Express REST API to manage employees and salaries.
Requirements: - GET /employees → list all employees - POST /employees → add employee with name, role, salary - PUT /employees/:id/salary → update salary - DELETE /employees/:id → remove employee
Add validation and comments for each endpoint.
Also suggest a proper data model.

Code:

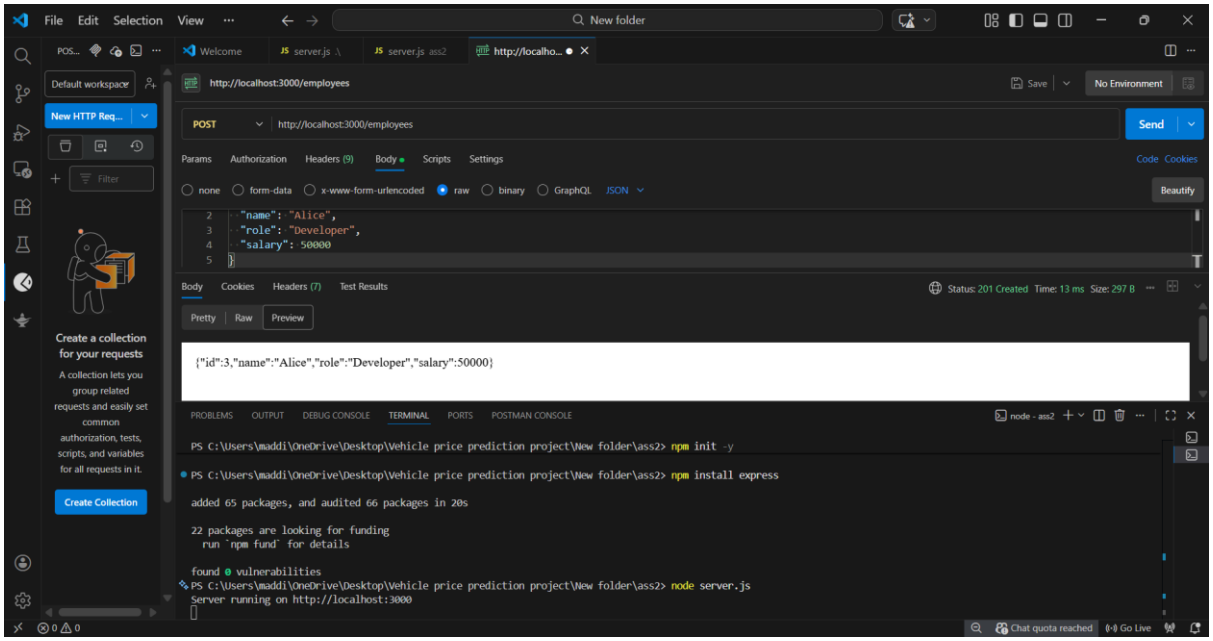




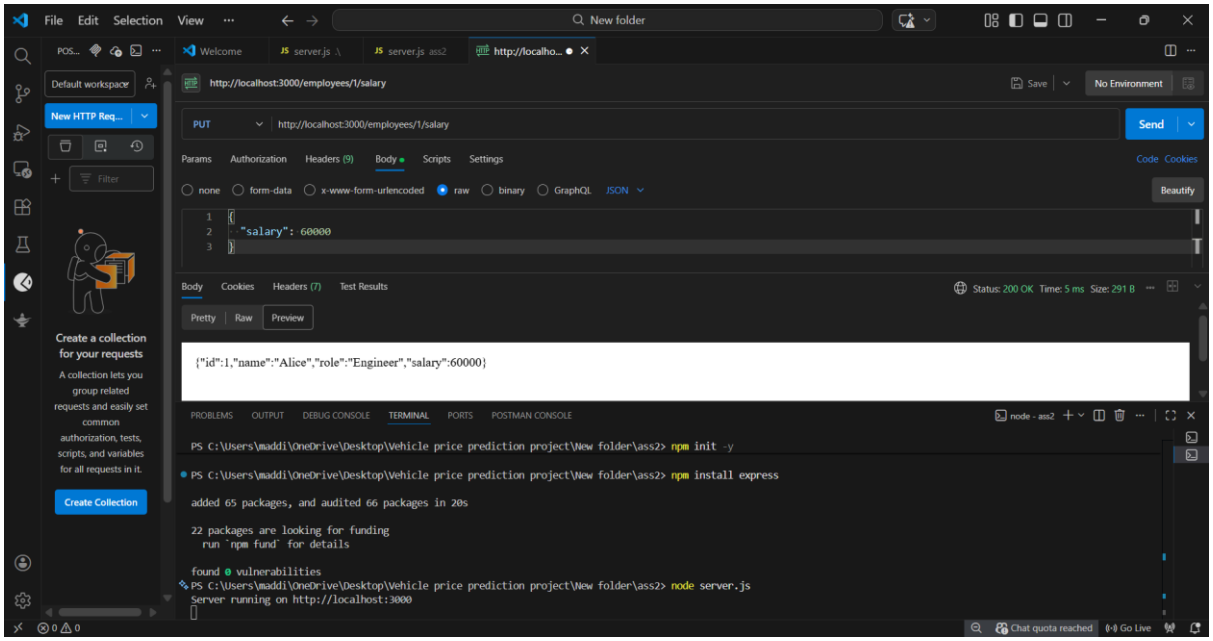
Get:


File Edit Selection View
New folder
Welcome
server.js
server.js ass2
http://localho...
Default workspace
New HTTP Req...
http://localhost:3000/employees
Save
No Environment
GET
http://localhost:3000/employees
Send
Params Authorization Headers (7) Body Scripts Settings
Code Cookies
Query Params
Key
Value
Body Cookies Headers (7) Test Results
Status: 200 OK Time: 18 ms Size: 357 B
Pretty Raw Preview
{"data":[{"id":1,"name":"Alice","role":"Engineer","salary":80000},{"id":2,"name":"Bob","role":"Manager","salary":95000}]}
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
PS C:\Users\maddi\OneDrive\Desktop\Vehicle price prediction project\New folder\ass2> npm init -y
PS C:\Users\maddi\OneDrive\Desktop\Vehicle price prediction project\New folder\ass2> npm install express
added 65 packages, and audited 66 packages in 20s
22 packages are looking for funding
run `npm fund` for details
found 0 vulnerabilities
PS C:\Users\maddi\OneDrive\Desktop\Vehicle price prediction project\New folder\ass2> node server.js
Server running on http://localhost:3000
Create a collection for your requests
A collection lets you group related requests and easily set common authorization, tests, scripts, and variables for all requests in it.
Create Collection

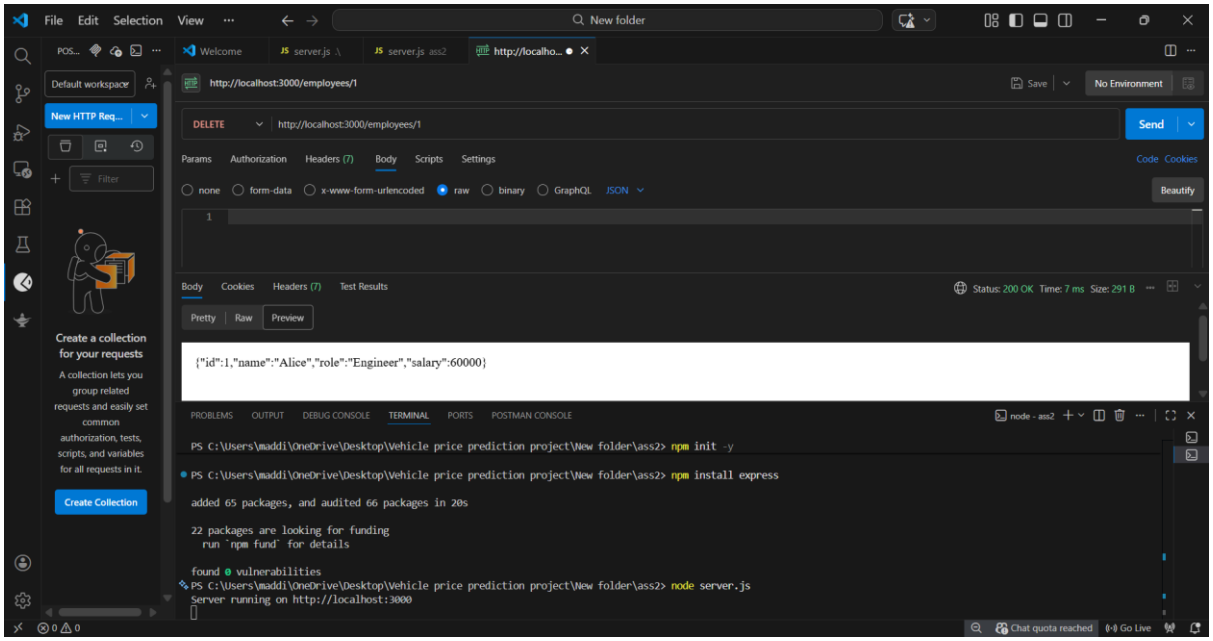
Post:


File Edit Selection View
New folder
Welcome
server.js
server.js ass2
http://localho...
Default workspace
New HTTP Req...
http://localhost:3000/employees
Save
No Environment
POST
http://localhost:3000/employees
Send
Params Authorization Headers (9) Body Scripts Settings
Code Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Beautify
"name": "Alice",
"role": "Developer",
"salary": 50000
Body Cookies Headers (7) Test Results
Status: 201 Created Time: 13 ms Size: 297 B
Pretty Raw Preview
{"id":3,"name":"Alice","role":"Developer","salary":50000}
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
PS C:\Users\maddi\OneDrive\Desktop\Vehicle price prediction project\New folder\ass2> npm init -y
PS C:\Users\maddi\OneDrive\Desktop\Vehicle price prediction project\New folder\ass2> npm install express
added 65 packages, and audited 66 packages in 20s
22 packages are looking for funding
run `npm fund` for details
found 0 vulnerabilities
PS C:\Users\maddi\OneDrive\Desktop\Vehicle price prediction project\New folder\ass2> node server.js
Server running on http://localhost:3000
Create a collection for your requests
A collection lets you group related requests and easily set common authorization, tests, scripts, and variables for all requests in it.
Create Collection

Put:



Delete:



Explanation:

his API is used to manage employee details and their salaries.

The **GET /employees** endpoint shows all employees.
The **POST /employees** endpoint adds a new employee with name, role, and salary.
The **PUT /employees/:id/salary** endpoint updates the salary of a specific employee.

The **DELETE /employees/:id** endpoint removes an employee from the system.

Basic validation is used to make sure correct data is entered before processing.

Task 3: Student Information REST API

Scenario:

A university wants to digitize student records for easy retrieval. You need to build

a Student Management API that allows:

• Adding a new student (name, roll number, course, year)

• Fetching details of all students

• Updating student information

• Deleting a student record

Implementation Instructions:

1. Use Node.js + Express (or Python Flask/Django REST) with GitHub

Copilot to auto-generate routes.

2. Store student data in an in-memory array or SQLite/PostgreSQL

database.

Test API endpoints with Postman.

Prompt:

```
Create a Student Management REST API using
Node.js and Express.
Features: - Add student (name, roll number, course,
year) - Get all students - Update student details -
Delete student
Store data in an in-memory array.
Add comments and clean structure.
```

Code:

File Edit Selection View Go ... Ass15-2

EXPLORER

OPEN EDITORS
- index.js

ASS15-2
- node_modules
- index.js
- package-lock.json
- package.json

OUTLINE
TIMELINE

index.js

```
const express = require('express');
const app = express();
const port = process.env.PORT || 3000;

app.use(express.json());

// In-memory student data store
// Data model for student:
// {
//   id: number,
//   name: string,
//   rollNumber: string,
//   course: string,
//   year: number
// }
let students = [
  { id: 1, name: 'Aisha Khan', rollNumber: 'R001', course: 'Compu
  { id: 2, name: 'David Lee', rollNumber: 'R002', course: 'Busine
];
let nextStudentId = 3;

/**
 * GET /students
 * Returns all students
 */
app.get('/students', (req, res) => {
  res.json({ data: students });
});

/**
 * POST /students
 * Add a new student (name, rollNumber, course, year)
 */
```

CHAT

NODE.JS EXPRESS REST API FOR EMPLO...

i have deleted index.js

Created index.js file with student API

Great progress—your app is back on track. Next I'll recreate `index.js` with student endpoints and run it.

Created index.js

Awesome—your student API is recreated with clean structure and full CRUD endpoints. Next I'll run and quickly verify it.

Running `cd "c:\Users\Siri Chunchu\AIAssisted-1281\Ass15-2"...`

```
cd "c:\Users\Siri Chunchu\AIAssisted-1281\Ass15-2"; npm.cmd start
```

index.js

Auto

Local Default Approvals

0 0 Ln 1, Col 1 Spaces: 4 UTF-8 CRLF JavaScript Go Live Prettier

File Edit Selection View Go ... Ass15-2

POSTMAN

Default workspace

New HTTP Request

Filter

Create a collection for your requests

A collection lets you group related requests and easily set common authorization, tests, scripts, and variables for all requests in it.

Create Collection

index.js http://localhos...

http://localhost:3000/students Save No Environment

GET http://localhost:3000/students Send

Params Authorization Headers (7) Body Scripts Settings Code Cookies

Body Cookies Headers (7) Test Results Status: 200 OK Time: 9 ms Size: 584 B

Pretty Raw Preview JSON


```
{
    "data": [
        {
            "id": 1,
            "name": "Sana Lee",
            "rollNumber": "RN1001",
            "course": "Computer Science",
            "year": 2,
            "createdAt": "2026-03-18T07:52:22.181Z",
            "updatedAt": "2026-03-18T07:52:22.182Z"
        },
        {
            "id": 2,
            "name": "Rahul Verma",
            "rollNumber": "RN1002",
            "course": "Mechanical Engineering",
            "year": 3,
            "createdAt": "2026-03-18T07:52:22.182Z",
            "updatedAt": "2026-03-18T07:52:22.182Z"
        }
    ]
}
```


CHAT

NODE.JS EXPRESS REST API FOR EMPLO...

postman urls to do

Postman URLs to Use

Assuming your server is running at `http://localhost:3000`:

1. Get all students
   - `GET http://localhost:3000/students`
2. Add student
   - `POST http://localhost:3000/students`
   - Body (JSON):

```
{
  "name": "Aisha Khan",
  "rollNumber": "RN1003",
  "course": "Computer Science",
  "year": 1
}
```

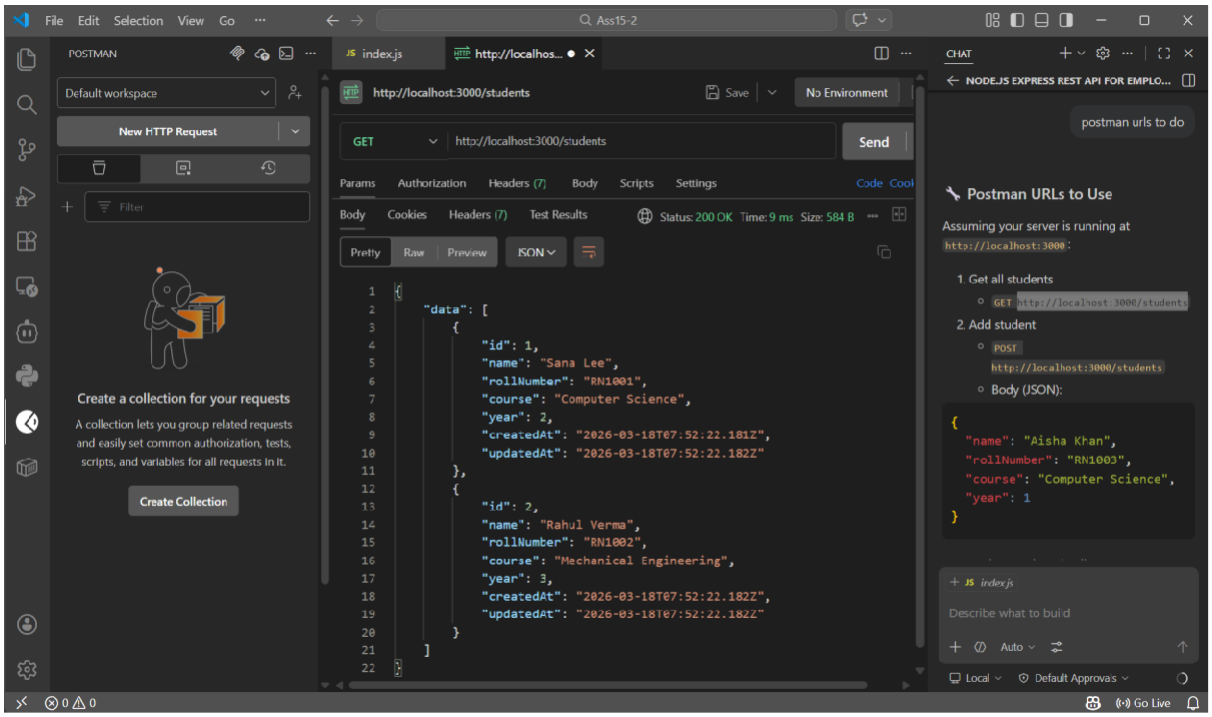
index.js

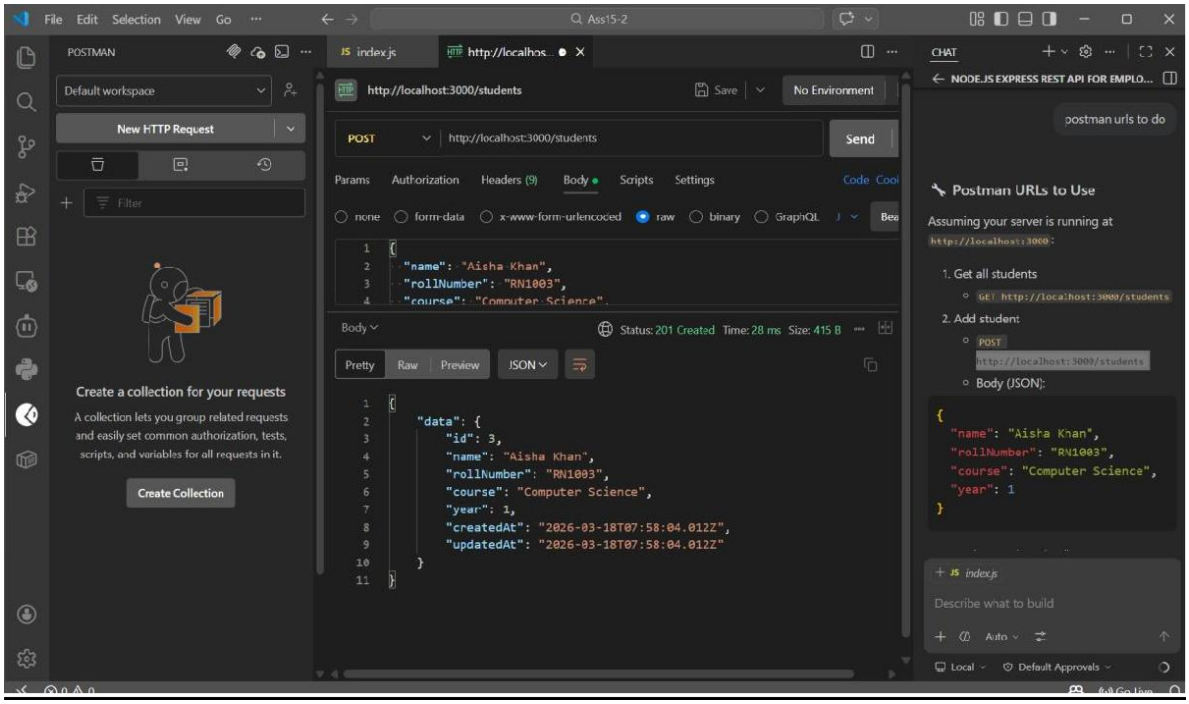
Describe what to build

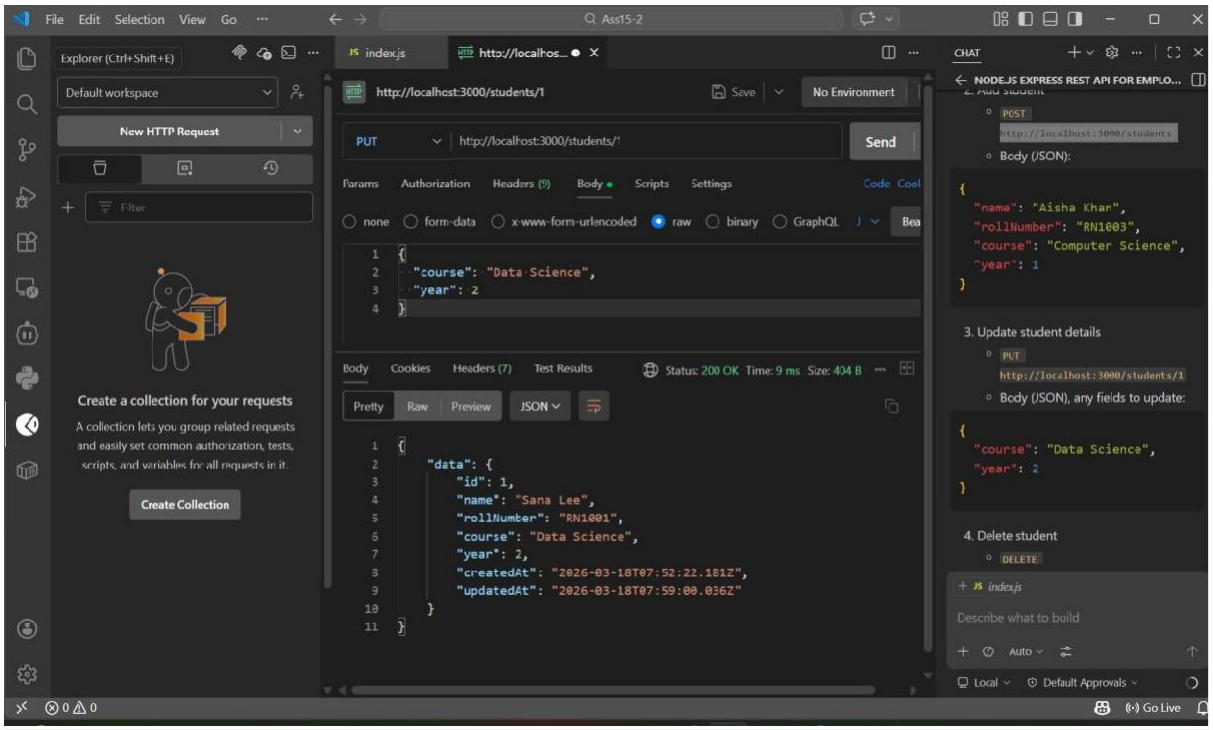
Auto

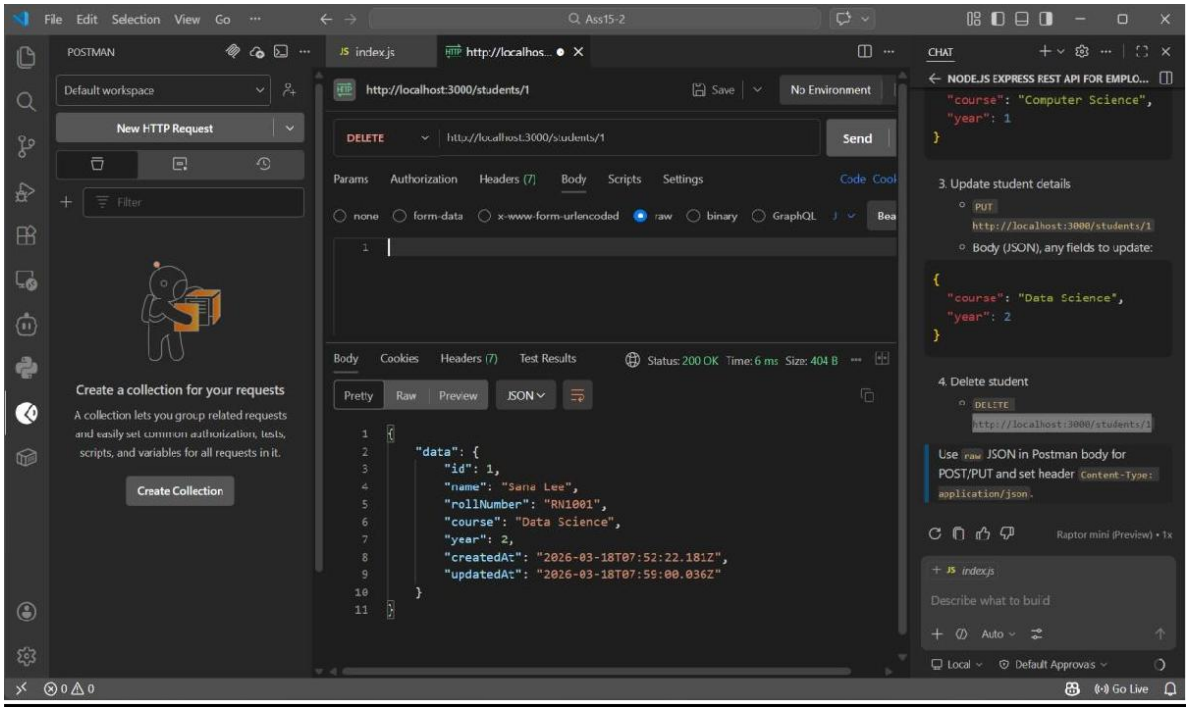
Local Default Approvals

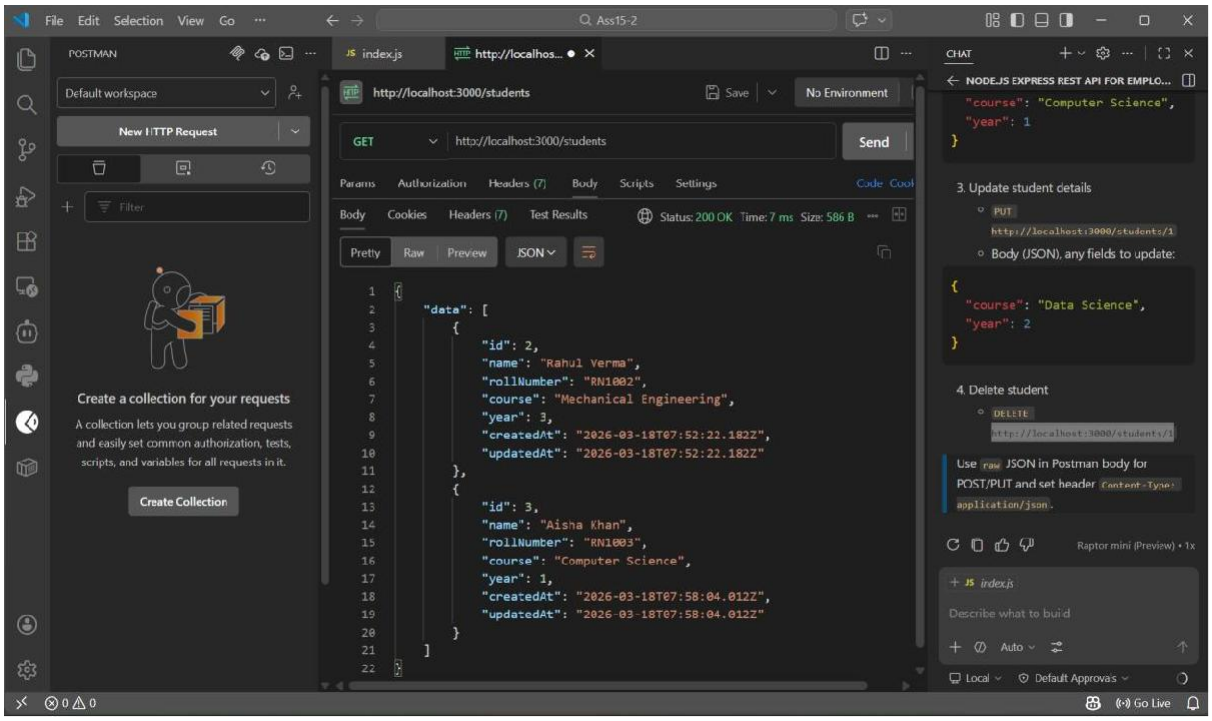
0 0 Go Live


File Edit Selection View Go
Ass15-2
POSTMAN
Default workspace
New HTTP Request
Filter
Create a collection for your requests
A collection lets you group related requests and easily set common authorization, tests, scripts, and variables for all requests in it.
Create Collection
index.js
http://localhos...
http://localhost:3000/students
Save
No Environment
GET
http://localhost:3000/students
Send
Params Authorization Headers (7) Body Scripts Settings
Code Cool
Body Cookies Headers (7) Test Results
Status: 200 OK Time: 9 ms Size: 584 B
Pretty Raw Preview JSON
{
"data": [
{
"id": 1,
"name": "Sana Lee",
"rollNumber": "RN1001",
"course": "Computer Science",
"year": 2,
"createdAt": "2026-03-18T07:52:22.181Z",
"updatedAt": "2026-03-18T07:52:22.182Z"
},
{
"id": 2,
"name": "Rahul Verma",
"rollNumber": "RN1002",
"course": "Mechanical Engineering",
"year": 3,
"createdAt": "2026-03-18T07:52:22.182Z",
"updatedAt": "2026-03-18T07:52:22.182Z"
}
]
}
CHAT
NODE.JS EXPRESS REST API FOR EMPLO...
postman urls to do
Postman URLs to Use
Assuming your server is running at http://localhost:3000:
1. Get all students
GET http://localhost:3000/students
2. Add student
POST http://localhost:3000/students
Body (JSON):
{
"name": "Aisha Khan",
"rollNumber": "RN1003",
"course": "Computer Science",
"year": 1
}
index.js
Describe what to build
Auto
Local
Default Approvals
Go Live


File Edit Selection View Go
Ass15-2
POSTMAN
Default workspace
New HTTP Request
Filter
Create a collection for your requests
A collection lets you group related requests and easily set common authorization, tests, scripts, and variables for all requests in it.
Create Collection
index.js
http://localhos...
http://localhost:3000/students
Save
No Environment
POST
http://localhost:3000/students
Send
Params Authorization Headers (9) Body Scripts Settings
Code Cool
none form-data x-www-form-urlencoded raw binary GraphQL
Body
Status: 201 Created Time: 28 ms Size: 415 B
Pretty Raw Preview JSON
{
"data": {
"id": 3,
"name": "Aisha Khan",
"rollNumber": "RN1003",
"course": "Computer Science",
"year": 1,
"createdAt": "2026-03-18T07:58:04.012Z",
"updatedAt": "2026-03-18T07:58:04.012Z"
}
}
CHAT
NODE.JS EXPRESS REST API FOR EMPLO...
postman urls to do
Postman URLs to Use
Assuming your server is running at http://localhost:3000:
1. Get all students
GET http://localhost:3000/students
2. Add student
POST http://localhost:3000/students
Body (JSON):
{
"name": "Aisha Khan",
"rollNumber": "RN1003",
"course": "Computer Science",
"year": 1
}
index.js
Describe what to build
Auto
Local
Default Approvals


PUT
http://localhost:3000/students/1
Send
Status: 200 OK Time: 9 ms Size: 404 B
"course": "Data Science",
"year": 2
"data": {
"id": 1,
"name": "Sana Lee",
"rollNumber": "RN1001",
"course": "Data Science",
"year": 2,
"createdAt": "2026-03-18T07:52:22.181Z",
"updatedAt": "2026-03-18T07:59:00.036Z"
3. Update student details
Body (JSON), any fields to update:
4. Delete student


DELETE
http://localhost:3000/students/1
Send
Status: 200 OK Time: 6 ms Size: 404 B
"data": {
"id": 1,
"name": "Sana Lee",
"rollNumber": "RN1001",
"course": "Data Science",
"year": 2,
"createdAt": "2026-03-18T07:52:22.181Z",
"updatedAt": "2026-03-18T07:59:00.036Z"
3. Update student details
4. Delete student



Explanation:

This API is designed to manage student records in a university system. The **POST /students** endpoint adds new student details.

The **GET /students** endpoint retrieves all student records.

The **PUT /students/:id** endpoint updates existing student information. The **DELETE /students/:id** endpoint deletes a student record.

Data is stored in an in-memory array for simplicity.

Task 4: Student Information REST API

Scenario:

A university wants to digitize student records for easy retrieval. You need to build

a Student Management API that allows:

- Adding a new student (name, roll number, course, year)
- Fetching details of all students
- Updating student information
- Deleting a student record

Implementation Instructions:

1. Use Node.js + Express (or Python Flask/Django REST) with GitHub Copilot to auto-generate routes.

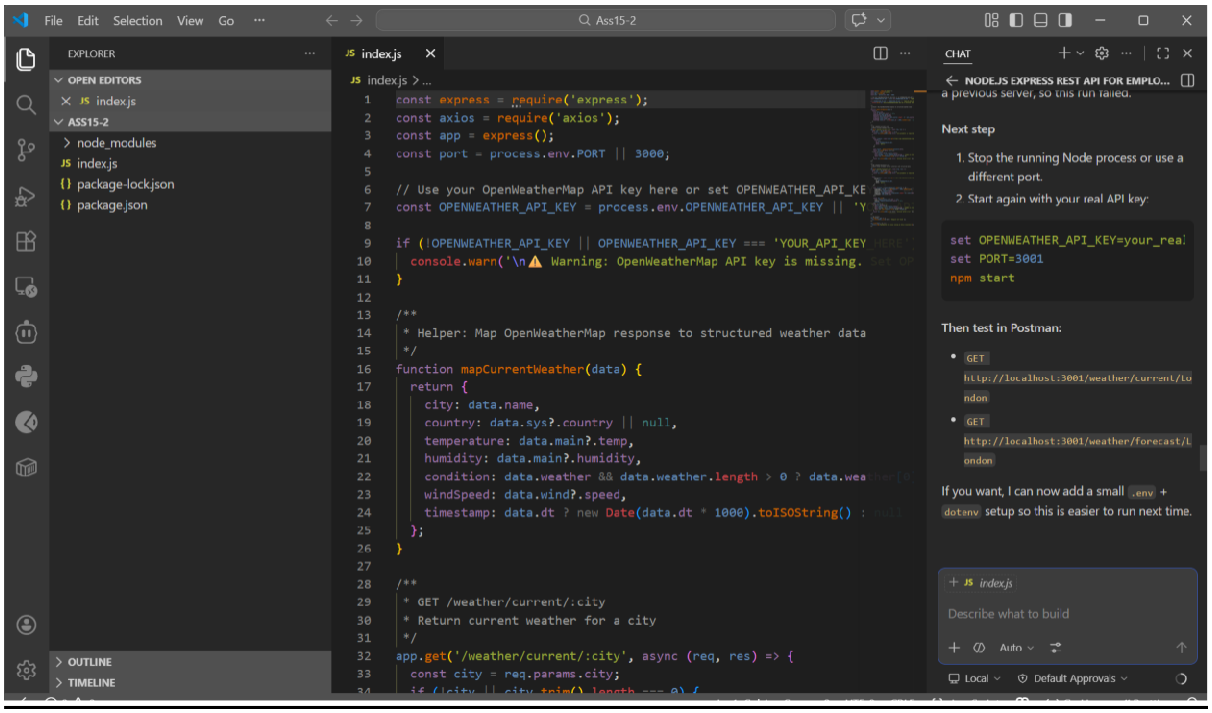
2. Store student data in an in-memory array or SQLite/PostgreSQL database.

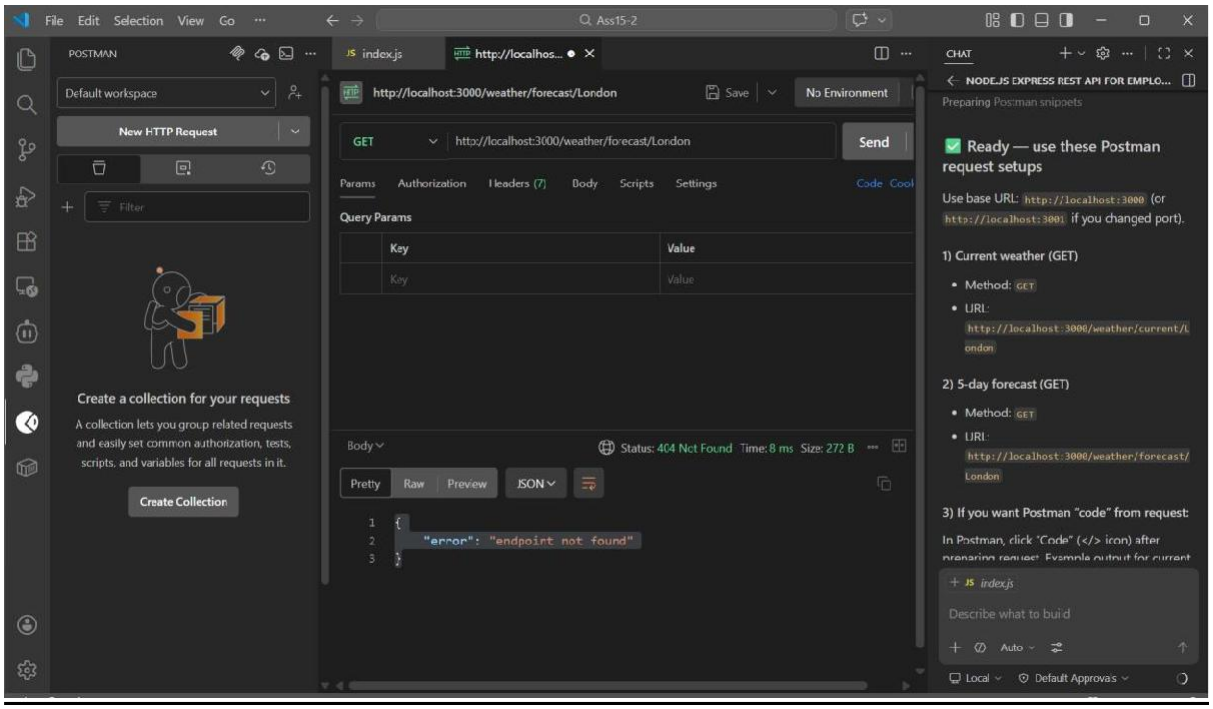
Test API endpoints with Postman.

Prompt:

Create a Node.js Express API that integrates with OpenWeatherMap API. Endpoints:
- /weather/current/:city → return temperature, humidity, condition
- /weather/forecast/:city → return 5-day forecast Use axios for API calls and return structured JSON. Include error handling and comments.

Code:





Explanation:

This API integrates with an external service (OpenWeatherMap) to fetch real-time weather data.

The /weather/current/:city endpoint retrieves current weather details such as temperature, humidity, and condition.

The /weather/forecast/:city endpoint provides forecast data

Axios is used to make HTTP requests to the external API, and structured JSON responses are returned to the user.

Task 5: Delete Item

Scenario:

Instructions:

• Use AI to create a DELETE endpoint to remove an item by index or ID.

Starter Code:

```
# DELETE /items/<int:index>

@app.route('/items/<int:index>', methods=['DELETE'])

def delete_item(index):

if index < 0 or index >= len(items):

return jsonify({"error": "Item not found"}), 404

removed_item = items.pop(index)

return jsonify({"message": "Item deleted", "item": removed_item})
```

Prompt:

Create a Flask API endpoint to delete an item from a list by index.
Requirements:
- DELETE /items/<int:index>
- Return error if index invalid
- Return deleted item in response Add proper comments.

Code:

```python
from flask import Flask, jsonify, request

app = Flask(__name__)

# In-memory list to store items
items = ["apple", "banana", "cherry"]


@app.route('/items', methods=['GET'])
def get_items():
    """
    GET /items
    Returns all items in the list
    """
    return jsonify({
        "success": True,
        "items": items
    }), 200


@app.route('/items', methods=['POST'])
def add_item():
    """
    POST /items
    Adds a new item to the list
    Expected JSON: { "item": "value" }
    """
    data = request.get_json()

    # Validate input
    if not data or "item" not in data or not data["item"]:
        return jsonify({
            "success": False,
```

```
Node.js v22.19.0
PS C:\Users\maddi\OneDrive\Desktop\Vehicle price prediction project\New folder\ass2> python3
```

DELETE http://127.0.0.1:5000/items/0

Params Authorization Headers (2) Body Pre-request Script Tests

none form-data x-www-form-urlencoded binary Graph

```
{
    "message": "Item deleted",
    "item": {
        "name": "Notebook",
        "price": 250
    }
}
```

Body Cookies Preseies Test (1) Test Results

Pretty Raw Preview Visualize JSON

```
{
```

GET http://127.0.0.1:5000/items

Params Authorization Headers (2) Body Pre-request Script Tests

none form-data x-www-form-urlencoded binary Graph

```
}
```

Body Cookies Preseies Test (1) Test Results

Pretty Raw Preview Visualize JSON

Explanation:

This API demonstrates how to delete an item from a list using a DELETE request. The DELETE /items/<index> endpoint removes an item based on its index.

It includes error handling to check if the index is valid.

If valid, the item is removed and returned in the response.